
A WIND-GATED LEARNED LAYER ON WIND-SCHEDULED MODEL PREDICTIVE PITCH CONTROL

WHAT A MODEL-BASED CONTROLLER AND A LEARNED ONE EACH CONTRIBUTE ABOVE
RATED, AND HOW THE TWO ARE REPORTED OVER A SITE WIND DISTRIBUTION

A PREPRINT

[authors]

September 23, 2026

ABSTRACT

Learned pitch controllers are usually reported against a default gain-scheduled PI, over a handful of wind speeds, as a single set mean. We take the same plant—collective pitch of the NREL 5 MW turbine in OpenFAST—and rebuild the comparison so that each layer of the controller is charged for what it actually adds. Three results follow. First, most of the reported margin of a learned controller over the classical baseline is a tuning margin: re-tuning one corner frequency of the baseline’s speed filter recovers 15.4 of it on a 12–24 m/s, class-B, 600 s protocol, and every comparison in this paper carries that tuned baseline. Second, a linear-parameter-varying MPC whose cost is correctly scaled, made offset-free by a slow output disturbance estimate and given a wind-scheduled tower weight, reaches 30.5 on winds nothing in the pipeline has seen—the strongest controller here—and its model error is absorbed by the estimator rather than by learning. Third, a bounded PPO residual is worth adding to that controller only where the controller leaves room: gated on the wind estimate so that it acts above 15–17 m/s and is exactly zero below, the residual improves *both* bases—the tuned baseline from 15.9 to 20.7 at $1.40\times$ its pitch travel, the MPC from 30.5 to 32.5—in 16 of 16 evaluations, each with a paired bootstrap interval above zero, on held-out and on fresh wind realisations alike. A language-model agent supervises the residual’s reward under fork verification; its measurable contribution is the *shape* of the reward, which holds the fatigue terms where a searched or fixed reward of the same level does not, and on the model-based base it is not distinguishable from a random search over its own vocabulary. Finally, the same results re-weighted over an IEC Rayleigh wind distribution change what may be claimed: the tower-fatigue reduction of the model-based layer doubles (22.1 % to 40.4 % for class I) while the gated layer’s fatigue contribution thins to nothing, because it acts in the tail. Both readings are reported.

1 Introduction

Above rated wind speed, collective pitch regulates rotor speed and electrical power while the tower and the blades accumulate fatigue. The classical solution is a gain-scheduled PI (GSPI) on generator speed with peak shaving [1, 2]; model predictive control (MPC) states the trade-off as a constrained optimisation [3–7]; and reinforcement learning (RL) has been proposed as a model-free alternative that learns the trade-off from simulation [8–11]. The three are usually compared as whole controllers on a small number of wind speeds, against whichever baseline the simulator ships with, and summarised as one number per metric.

That comparison hides two things, and this paper is about both. The first is *what the baseline was worth*. The controller a wind-energy toolbox ships is a reference implementation, not a tuned controller; part of the margin a learned controller reports is the margin of tuning, not of learning. The second is *where in the operating envelope a layer earns its margin*. A controller that improves the set mean can be improving one wind speed and damaging another, and a fatigue number aggregated over an equally weighted set of wind speeds is not the number a site sees, because the wind distribution is not uniform.

We therefore build the comparison as a ladder. Each rung is a controller that the next rung has to beat on the same 42 or 28 episodes of 600 s, at seven mean wind speeds from 12 to 24 m/s at IEC turbulence class B, with every percentage taken against the *same* paired GSPI episode: the shipped GSPI; the same GSPI with its speed-filter corner re-tuned; a three-state LPV-MPC with a correctly scaled cost; that MPC made offset-free by a slow output-disturbance estimate; the same MPC with a wind-scheduled tower weight; and finally a bounded learned residual on top of either of the two deployable bases. The learned layer is supervised by a language-model agent that writes its reward under fork verification, in the manner of Eureka and its successors [12–16].

Results. (i) Re-tuning one corner frequency of the baseline’s speed filter is worth 15.4 points of the objective—a substantial part of what a learned controller would otherwise be credited with. (ii) The wind-scheduled offset-free MPC reaches 30.5 on fresh wind realisations at $2.1 \times$ the baseline’s pitch travel, and its model error is best absorbed by its own estimator: a residual trained across $\pm 15\%$ coefficient error repairs only the underestimate side and costs 3–6 points at the exact model, while the estimator is flat. (iii) A residual gated on the wind estimate—zero below 15 m/s, full above 17—improves both bases in 16 of 16 evaluations with paired intervals above zero, on held-out and on fresh winds: $15.9 \rightarrow 20.7$ on the tuned baseline at $1.40 \times$ pitch travel, $30.5 \rightarrow 32.5$ on the MPC. Against the shipped GSPI, the full stack reduces power MSE by 51.5 %, generator-speed MSE by 58.0 % and tower-base fatigue by 18.9 % on winds nothing has seen. (iv) The agent’s contribution is the reward’s shape: on the tuned-PI base its rewards hold every fatigue term per wind speed where a random structural search of the same level does not; on the MPC base the two are indistinguishable. (v) Re-weighting the same episodes over an IEC Rayleigh distribution doubles the MPC’s tower-fatigue claim and removes the gated layer’s, which must both be stated.

Contributions. This paper’s contributions are measurements, not mechanisms; the gate, the weight schedule and the offset-free correction all have precedents (§2). (a) A protocol in which a learned layer is charged against a *tuned* classical baseline and a strong model-based one, over a wind range, per wind speed, with paired bootstrap intervals, and replicated on wind fields generated after the controllers were frozen. (b) The measurement of where a residual’s activation must close on this plant, and of how much of its apparent margin on a strong base is a redistribution across the envelope rather than a gain. (c) A separation of what the agent supervising the reward does (shape, load holding) from what it does not (level, and anything on a base that already holds the loads). (d) Site-weighted reporting of the same episodes, and the opposite consequences it has for the model-based and the learned layer’s claims.

2 Related work

MPC for wind-turbine pitch. Linear-parameter-varying and quasi-LPV formulations of the above-rated pitch problem are established [3, 4, 7], including preview variants [5] and robust ones [6, 17], and the practical obstacles to running them on a machine are documented [18]. Two elements of our controller exist in that literature and we claim neither as new. Scheduling the cost *weights* on an operating variable was introduced by Wintermeyer-Kallen et al. [19], who schedule Q and R on plant outputs that vary with wind and rotor speed to smooth the partial-to-full-load transition; what we schedule is a tower-fatigue weight *within* the full-load region, for a reason that only appears when terms are read per wind speed (§3.3). Offset-free MPC for wind energy with a disturbance observer is likewise established [20]; our contribution is not the formulation but the measurement of what the estimator is worth *against a learned residual* on the same plant and objective (§5). Closest to our model-error result is Anand and Bottasso [21], who repair the model of an economic MPC for the same turbine in the same simulator with an offline-trained network and report +9 % profit; we differ in that the repair here is online and recursive and the objective is a fixed four-metric regulation-and-fatigue set rather than a profit function.

RL for pitch and load control. Learned collective and individual pitch controllers are reported against a PI or GSPI baseline on a small set of wind speeds [8–11]; simultaneous tuning of collective and individual pitch is the classical counterpart [22]. What is rarely reported is what the baseline was worth: the corner frequency of the shipped speed filter in ROSCO follows a generic rule of thumb [2], which is exactly why re-tuning it is a fair comparison rather than a weakened one, and it is worth 15.4 points of our objective (§3.2).

Residual policies on a nominal controller. Learning a bounded correction on top of a classical controller is a standard construction, and restricting *when* the correction may act is part of that construction rather than an innovation of ours: Abbas et al. [23, 24] specialise residual policies by operating regime, Kim et al. [25] combine a model-based base carrying a disturbance observer with a state-dependent action bound that suppresses the residual inside a deadband, and Cao et al. [26] compose a learned policy with a safety-structured base. Our gate is the same mechanism with the wind-speed estimate as the gating variable; what we contribute is the evidence for *where* it must close on this

plant—measured per wind speed against a strong base—and the replication on wind fields generated after the controllers were frozen.

Language-model agents that design rewards and tune controllers. Eureka evolves reward code against a task score [12], Language-to-Rewards and Text2Reward map instructions to reward parameters and programs [13, 14], DrEureka extends the idea to domain randomisation [15], and agentic designers close the loop with semantic feed-back [16]; the construction has since reached control tasks directly [27–30], including reward-*weight* adaptation for a building energy plant [31]. Two of these bound our claim. First, weight adaptation is the comparison our ablations are built around: what we can attribute to the agent is the reward’s *shape*, not a weight the same search could have found (§5). Second, Chen et al. [32] show that open-weight models rank controller gains correctly only 31.8–53.2 % of the time and conclude that simulator-based validation remains necessary—which is the reason every proposal here is verified by a 30-episode fork before it is kept, and the reason we also report a random search over the agent’s own vocabulary under the same verification.

Evaluation conventions. Short-term damage-equivalent loads with Wöhler exponents 4 (tower) and 10 (blade) [33], aggregation over a Rayleigh wind distribution, and an actuator duty cycle are the field’s standard reporting instruments; rotor-effective wind-speed estimation, on which our gate depends, has its own literature and calibration [34]. Paired bootstrap intervals per wind speed are, as far as we found, not standard practice in this literature, and we adopt them because the controllers compared here are deterministic and the only source of variation is the wind realisation.

3 Setup

3.1 Plant, wind protocol, objective

The plant is the NREL 5 MW onshore turbine [35] in OpenFAST 4.2.1 [36], driven at a 10 ms control step by ROSCO 2.10.5 [2] through a patched ZeroMQ interface that exposes the full controller state. The pairing reference throughout is the shipped ROSCO GSPI with peak shaving on, every other load-mitigation feature off and constant generator torque above rated.

Wind protocol. Seven mean hub wind speeds, 12 to 24 m/s in steps of 2, IEC turbulence class B, TurbSim fields of 600 s of which the first 20 s are discarded as the start-up transient and the remaining 580 s are scored; no controller acts during the discarded window. Realisations with seeds 1–2 are the supervisor’s evaluation winds and the only winds any selection sees; seeds 3–6 (28 episodes) are held out; seeds 7–10 (28 episodes) were generated after the controllers were frozen and are seen by no training, no selection and no earlier version of this paper. Training and in-training selection use 150 s episodes of the same seven mean wind speeds at seeds 1–2, for cost; *every* reported number is a 600 s evaluation. Every controller is evaluated on the identical wind fields, and every percentage is taken against the GSPI episode on that same field, so the comparison is paired episode by episode and the only source of uncertainty is the wind realisation.

Objective. We adopt the metric set of a published RL pitch-control study [11] and reduce it to one scalar,

$$J = \frac{1}{4} \sum_{m \in \{P, \omega, T, B\}} \text{clip}_{\pm 100}(\Delta m [\%]) - 20 \max(0, E_{\text{loss}} [\%] - 1), \quad (1)$$

the mean of the per-episode percentage reductions, against paired GSPI, of power MSE, generator-speed MSE (both on above-rated steps, labelled by hub wind speed whatever the controller does), tower-base fore-aft damage-equivalent load (Wöhler exponent 4) and blade-root out-of-plane damage-equivalent load (exponent 10), minus a penalty for energy loss beyond 1 %. Fatigue is rainflow-counted [33, 37]. Because a set mean can hide a redistribution across the envelope, training and selection use a per-wind variant C_w , which subtracts 20 points for every metric that falls more than 1 % below the base at *any* mean wind speed, and every result is also reported per wind speed.

Statistics. All controllers here are deterministic given the wind field, so the uncertainty reported is over wind realisations, not over control noise: differences between two controllers are paired stratified bootstraps over episodes (10 000 resamples, strata = mean wind speed), quoted as 95 % intervals of the difference in J . Across training seeds we report every seed rather than a mean over few seeds.

3.2 Baselines: what tuning is worth

The shipped GSPI is the reference of (1) and scores 0 by construction. Its speed-filter corner frequency is the part of it that a deployment would tune; the shipped value is 1.57 rad/s. A one-dimensional search over that corner and the

proportional gain, selected on the supervisor winds only, gives 3.42 rad/s with the gain scaled by 1.13. This *tuned* baseline scores 15.4 on the 42-episode range set at $1.14\times$ the shipped controller’s pitch travel (Table 1) — a margin that a learned controller compared against the shipped default would have collected for itself. Every comparison in this paper carries the tuned baseline. For completeness, enabling ROSCO’s own tower-fore-aft damper on the shipped controller is worth -0.05 points on this protocol.

3.3 The model-based controller

A three-state LPV-MPC (rotor speed, first tower fore-aft mode and its velocity) re-linearised every 0.1 s from the C_P/C_T tables, horizon 2 s, solved with OSQP [38] in 3.6 ms, commanding pitch inside the live peak-shaving floor and the rate limit, with the wind held at ROSCO’s low-passed estimate (no preview) and native torque. Three design points matter.

Cost scaling. The three cost terms—speed error, tower-top velocity, pitch increment—are each normalised to $O(1)$ at their typical magnitude. With the speed error normalised by rated speed instead, that term is $O(10^{-5})$ at a typical 0.5 % error while the tower term is $O(1)$; any tower weight then outweighs regulation by four orders of magnitude, the optimiser feathers the rotor, and the best such MPC carries no tower term at all and loses tower fatigue. The published verdict that MPC trades tower fatigue for regulation is, on this plant, an artefact of that scaling.

Offset-free correction. A slow (5 s) output-disturbance estimate on the speed channel makes the controller offset-free, a standard construction in wind-energy MPC [20]. It is what absorbs model error: with the power and thrust coefficients 5 % low in the internal model the nominal MPC loses 0.7 points over the range, the offset-free one loses nothing, and a recursive-least-squares variant is flat within 0.3 points over $\pm 15\%$ coefficient error.

Wind-scheduled tower weight. The tower weight that is right near rated is wrong at 22–24 m/s, where the compensated controller’s tower term turns negative. Scheduling it on the wind estimate—the nominal weight 3 below 18 m/s, ramping linearly to 6 by 22 m/s—keeps every term non-negative per wind speed, at a cost of 0.6 points of J . Scheduling MPC cost weights on an operating variable is due to Wintermeyer-Kallen et al. [19]; what is specific here is the variable, the region—inside full load rather than across the partial-to-full-load transition—and the per-wind-speed measurement that motivates it. This *scheduled MPC* is the strongest controller here and the base the learned layer is stacked on.

3.4 The learned layer and its wind gate

Two PPO [39] policies—one per operating region, selected by a rule on the base controller’s own pitch command—add a bounded collective-pitch offset ($|\Delta\beta| \leq 0.05$ rad, second-order damped) to the base controller’s target; the pitch limits apply to the sum, so with a zero residual the controller *is* the base and the episode-0 policy is exactly the base’s behaviour. Observations are generator-speed error and its rate, measured pitch, hub wind speed, blade-root out-of-plane moment, tower-top fore-aft acceleration and the base’s target. $\gamma = 0.998$ (a 0.99 discount is myopic with respect to the ≈ 3 s tower mode and every method fails with it), GAE $\lambda = 0.98$ [40], 64×64 tanh networks, 300 episodes per run, critic targets standardised by a running return-scale estimate [41].

The gate. The residual’s increment is multiplied by $g(\hat{v})$, zero below 15 m/s, one above 17 m/s and linear between, on a 10 s low-pass of the controller’s wind-speed estimate:

$$\Delta\beta_t \leftarrow g(\hat{v}_t) \Delta\beta_t, \quad g(v) = \text{clip}((v - 15)/2, 0, 1). \quad (2)$$

The gate is present during training, so selection sees it. Restricting when a residual may act is an established construction—by operating regime [23], by a state-dependent bound that suppresses the correction inside a deadband on a base that already carries a disturbance observer [25], or by composition with a safety-structured base [26]—and we claim neither the mechanism nor the idea, only the gating variable and the evidence for where it must close. That evidence is measured, not assumed: ungated, the same layer gains 12–16 points of regulation above 18 m/s and damages the 12–14 m/s bins, where a strong base already holds every term. Gated, the whole learning budget goes where the base leaves room, and below the gate the controller is provably the base (Fig. 2). The gate depends on a rotor-effective wind-speed estimate, whose accuracy is a controller property with its own literature [34]; here it is the base controller’s own low-passed estimate, so the gate adds no sensing requirement.

The fixed reward, used as a control.

$$r_t = \mathbf{1}[\text{Region 2}] w_P \left(\frac{P}{P_{\text{base}}} - 1 \right) - \mathbf{1}[v_{\text{hub}} > v_{\text{rated}}] w_\omega \left(\frac{\delta\omega}{0.005} \right)^2 - \lambda_T \ell_T - \lambda_B \ell_B - \lambda_A (\Delta\beta/\kappa)^2, \quad (3)$$

with fatigue proxies ℓ_T, ℓ_B given by the growth of the 10 s trailing peak-to-peak range of tower acceleration and blade-root moment (≈ 1 per step under GSPI; the instantaneous moment and its increment both destabilise training), and the speed term gated by the wind label—the same subset on which the objective’s regulation metrics are scored. Its six coefficients were tuned once, on the supervisor winds, against (1), and are identical in every run that uses it.

Table 1: The controller ladder on the range set (42 episodes of 600 s, 12–24 m/s, class B, wind seeds 1–6), every row against the same paired shipped-GSPI episodes. Terms are percentage reductions; duty is pitch travel relative to GSPI. The tuned baseline is what a learned controller must beat.

controller	J	power MSE	speed MSE	tower DEL	blade DEL	duty $\times$ GSPI
shipped GSPI (identity check)	0.00	0.0	0.0	0.0	0.0	1.00
+ ROSCO tower damper	−0.05	−0.3	−0.6	0.5	0.2	1.01
tuned GSPI baseline	15.39	23.2	29.2	5.4	3.8	1.14
nominal LPV-MPC	28.48	42.0	46.9	20.9	4.1	2.01
with $C_P/C_T \times 0.95$ in its model	27.78	43.5	45.0	19.0	3.7	2.06
offset-free LPV-MPC	29.50	47.6	52.7	14.4	3.3	1.97
with $C_P/C_T \times 0.95$ in its model	29.50	48.1	53.5	13.3	3.1	2.01

Table 2: The wind-gated learned layer, each row against its own base on the same episodes (paired stratified bootstrap over episodes, 95 % interval of the difference). Held out = wind seeds 3–6; fresh = seeds 7–10, generated after the controllers were frozen. All sixteen differences are above zero. “worst per wind” is the largest shortfall of any of the four terms against the base at any single mean wind speed, on the fresh set.

base	layer	J held out (Δ)	J fresh (Δ)	duty $\times$ GSPI	worst per wind
tuned GSPI baseline	—	15.24	15.93	1.12–1.15	—
	agent reward, seed 0	19.35 (+4.10 [3.96, 4.26])	20.14 (+4.21 [3.95, 4.45])	1.45	−1.0 tower @ 18
	agent reward, seed 1	19.52 (+4.27 [4.07, 4.49])	20.19 (+4.26 [3.98, 4.55])	1.42	−2.7 tower @ 18
	agent reward, seed 2	20.04 (+4.79 [4.58, 5.02])	20.68 (+4.76 [4.45, 5.06])	1.40	−1.4 tower @ 18
scheduled MPC	—	28.95	30.46	2.08–2.14	—
	agent reward, seed 0	30.60 (+1.65 [1.41, 1.90])	31.98 (+1.52 [1.30, 1.75])	2.69	−1.7 tower @ 24
	agent reward, seed 1	31.22 (+2.27 [1.99, 2.55])	32.54 (+2.08 [1.53, 2.63])	2.90	−1.6 tower @ 18
	agent reward, seed 2	31.10 (+2.15 [1.92, 2.38])	32.42 (+1.96 [1.65, 2.26])	3.10	−2.0 tower @ 24
	fixed reward, seed 0	31.20 (+2.24 [1.96, 2.53])	32.24 (+1.78 [1.54, 2.03])	3.06	−5.9 tower @ 20
	fixed reward, seed 1	30.05 (+1.10 [0.84, 1.35])	31.62 (+1.16 [0.69, 1.65])	2.38	−1.9 power @ 16

3.5 The supervising agent

Every 30 episodes the policy is evaluated on the supervisor winds and a reasoning language model (OpenAI-compatible, JSON mode, about five calls per run) receives the training diagnostics, that evaluation, the decision history with outcomes, and the objective’s definition. It returns $K = 3$ candidate *reward expressions*: sandboxed Python over (region, wind label, speed error, power ratio, both fatigue proxies, actuation) that replace (3). Each candidate is trained for a 30-episode fork from the current checkpoint, the forks are evaluated on the supervisor winds, and the best fork continues; a guardrail rolls the run back to its best checkpoint when the objective drops by more than five points or energy loss exceeds 1 %. Two controls run through the identical loop: the fixed reward (3) with no agent, and a random search over a fixed grammar of the agent’s own term shapes with log-uniform coefficients, verified by the same forks.

4 Results

4.1 The ladder

Table 1 reads from the bottom of the ladder up. Tuning one corner of the shipped baseline is worth 15.4 points at $1.14\times$ its pitch travel; the tower damper the toolbox provides is worth nothing on this protocol. The MPC is worth another 13 points on top of the tuned baseline, at twice the baseline’s pitch travel, and it buys them from regulation ($23 \rightarrow 42\%$ power MSE) and from tower fatigue ($5 \rightarrow 21\%$). Offset-free correction is a further point of J but is a trade—more regulation, less tower fatigue—and it is what makes the controller indifferent to a 5 % aerodynamic coefficient error, which costs the nominal controller 0.7 points. The scheduled variant of §3.3, evaluated on the held-out and fresh sets below, is the one used as the learned layer’s base.

4.2 The gated layer improves both bases, on winds nothing has seen

Table 2 is the main result. On the tuned baseline the gated layer adds +4.2 to +4.8 points, on the scheduled MPC +1.5 to +2.1, and every one of the sixteen paired intervals—eight runs $\times$ two wind sets—excludes zero. The held-out and the fresh sets agree to within 0.3 points on every run, which is the property that matters: the fresh wind fields

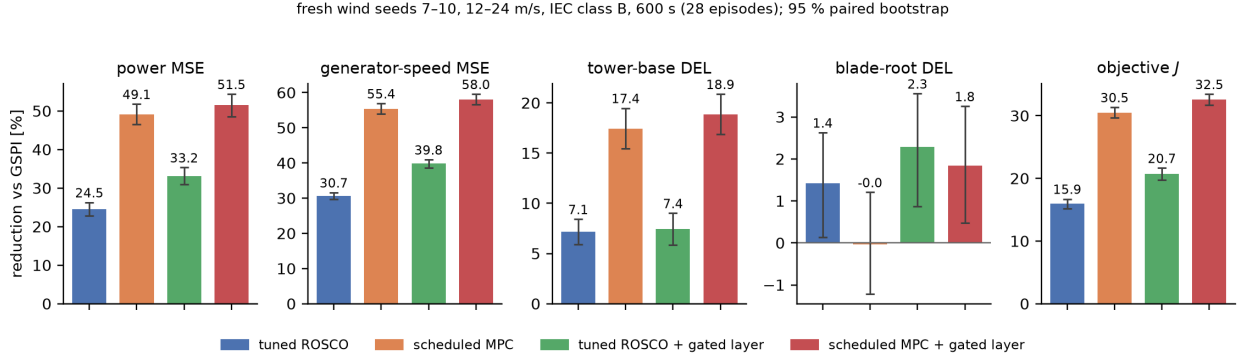


Figure 1: The four terms of the objective and J itself for each controller of the stack, on the fresh wind seeds (28 episodes of 600 s), against the shipped GSPI. Error bars are 95 % paired stratified bootstraps over episodes.

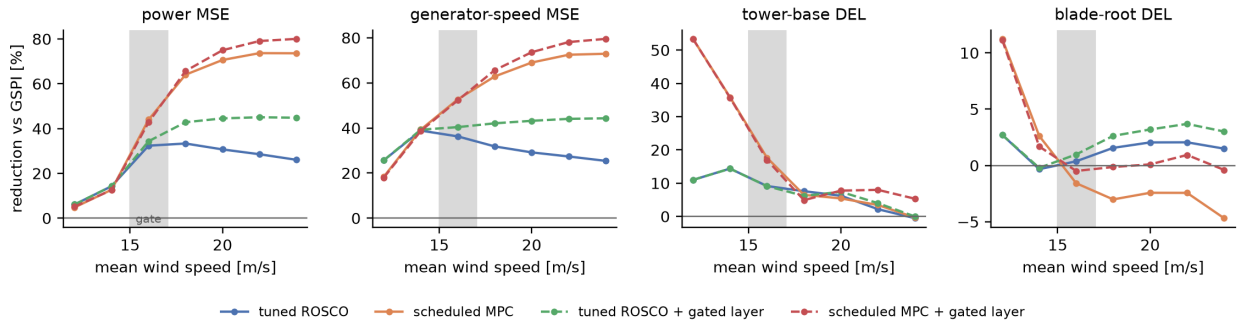


Figure 2: Where each layer acts. The four terms per mean wind speed on the fresh seeds; shaded band = the gate ramp of (2). Below the gate each gated curve coincides with its base, by construction. Above it the layer adds 10–12 points of regulation on either base. The tower-fatigue panel also explains §4.4: the model-based layer earns its tower reduction at 12–14 m/s, the learned layer earns nothing there.

were generated after the controllers were frozen, so the agreement is not a selection artefact. Against the shipped GSPI, the strongest stack (scheduled MPC + gated layer) reaches power MSE -51.5% , generator-speed MSE -58.0% , tower-base DEL -18.9% and blade-root DEL -1.8% on the fresh set (Fig. 1). Against the *tuned* baseline the same row is -36% , -39% and -13% .

The cheap configuration is worth stating separately: the tuned baseline plus the gated layer needs no model at all, reaches 20.7, and costs $1.40\times$ the shipped controller’s pitch travel, against $2.08\times$ for the MPC alone (Fig. 4).

4.3 Where the margin comes from

Fig. 2 resolves the same episodes by mean wind speed and is the reason for the gate. Below 15 m/s each gated curve lies exactly on its base’s, because the residual is multiplied by zero—a property of the construction, visible as a check. Above the gate the layer adds 10–12 points of power- and speed-MSE reduction on either base. The fatigue panels show the division of labour: the scheduled MPC takes tower-base fatigue down by 53 % at 12 m/s and 36 % at 14 m/s and by almost nothing at 22–24 m/s, while the learned layer’s tower contribution is confined to the high winds where it is active, and it repairs the MPC’s negative blade-fatigue term above 18 m/s (from -5% to ≈ 0).

4.4 The same result over a site wind distribution

Every number so far is the mean over an equally weighted set of mean wind speeds, which is the objective’s definition and the fair way to compare controllers per wind speed. A site does not see that distribution. Aggregating the identical episodes over an IEC Rayleigh distribution (Weibull shape 2, annual mean $0.2 V_{\text{ref}}$) with Palmgren–Miner damage summation gives Table 3 and Fig. 3, and the two readings differ in opposite directions.

The model-based layer’s fatigue claim grows: tower-base equivalent load -22.1% equally weighted becomes -40.4% for class I and -46.1% for class III, because tower fore-aft fatigue is dominated by the near-rated bins, which carry

Table 3: The same fresh-seed episodes, aggregated over an IEC Rayleigh wind distribution (weights renormalised over the evaluated 12–24 m/s bins; fatigue aggregated as $\text{DEL}_{\text{eq}} = (\sum_i w_i \text{DEL}_i^m / \sum_i w_i)^{1/m}$, regulation as a weighted mean), next to the equally weighted set mean used elsewhere. Reduction against paired GSPI, in %.

controller	power MSE	speed MSE	tower DEL	blade DEL
	Rayleigh / set mean	Rayleigh / set mean	Rayleigh / set mean	Rayleigh / set mean
tuned GSPI baseline	7.7 / 10.4	29.4 / 28.4	11.3 / 7.9	1.4 / 1.5
scheduled MPC	7.4 / 15.3	42.3 / 60.9	40.4 / 22.1	−1.8 / −0.5
tuned baseline + gated layer	8.1 / 12.7	35.1 / 40.9	11.4 / 8.1	2.4 / 2.4
scheduled MPC + gated layer	7.7 / 16.1	43.7 / 64.9	40.6 / 23.4	0.8 / 1.7

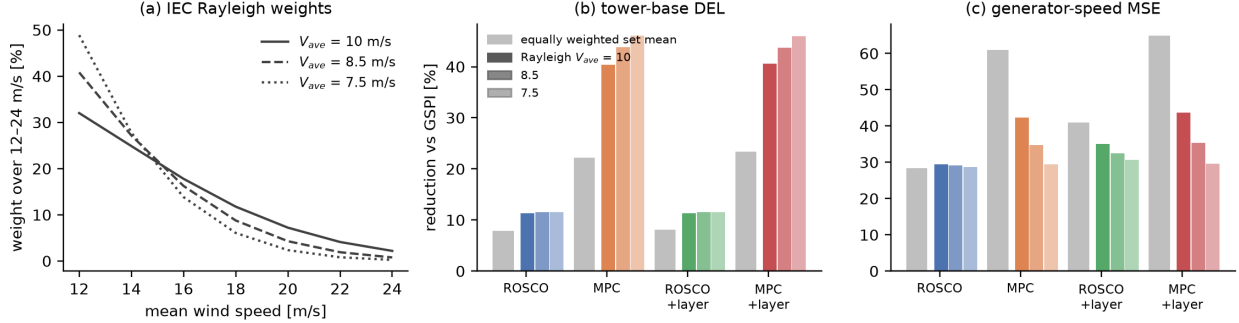


Figure 3: (a) Rayleigh weights over the evaluated bins for IEC classes I–III. (b, c) The tower-fatigue and speed-regulation reductions of the same controllers, equally weighted (grey) and Rayleigh-weighted at three annual mean wind speeds. Weighting doubles the model-based controller’s fatigue claim and removes the learned layer’s.

both the largest loads and the largest probability, and that is exactly where the scheduled tower weight acts. Against the tuned baseline the scheduled MPC is +32.8 % (class I) to +39.0 % (class III) of equivalent tower load.

The learned layer’s fatigue claim vanishes: gated above 15 m/s, it acts in the tail of the distribution, and on the tuned baseline its weighted tower contribution is 0.0 ± 0.1 points where the set mean credited it with +4.4. What survives the weighting is +0.4 power MSE, +7–8 speed MSE and +0.8 to +2.0 blade DEL at class I, and about a third of that at class III; on the MPC base, +0.3, +2.4, +0.3 tower and +2.6 blade.

Neither aggregation is a substitute for the other. Reporting only the set mean understates the model-based controller’s fatigue result by half; reporting only the site-weighted number understates the learned layer, whose margin sits in the winds that are rare but carry the largest regulation error and the largest pitch duty. We report both, and recommend that comparisons of learned pitch controllers do the same.

4.5 What the layer costs the actuator

Every margin above is bought with pitch activity, and it is reported with every row: the tuned baseline uses $1.14\times$ the shipped controller’s pitch travel, the MPC rows $1.97\text{--}2.14\times$, the gated layer a further 25–45 % on top of its base (Fig. 4). In absolute terms, as the actuator duty cycle $\text{ADC} = \frac{1}{T} \int |\dot{\beta}| / \dot{\beta}_{\text{max}} dt$ with the controller’s rate limit $\dot{\beta}_{\text{max}} = 10^\circ/\text{s}$, the fresh-seed values are 0.048 for the shipped GSPI, 0.054 tuned, 0.100 for the scheduled MPC, 0.067–0.070 for the tuned baseline with the gated layer and 0.129–0.149 for the MPC with it: every controller here is an order of magnitude below actuator saturation, and the duty ratio, not the absolute duty, is the meaningful cost. The activity is broadband rather than concentrated at the blade-passing frequency: a 3P notch on the command removes the spectral line, cuts travel by 2–3 % and costs 1.5–2.4 points of J . A search over the MPC’s own weights at fixed duty has its knee at about $3\times$ the shipped baseline’s travel, and a controller selected at that knee keeps the same J at 38–41 % of the actuation of the J -selected point, at a cost of 3 points of blade fatigue.

5 Ablations: what the agent contributes, and what it does not

The reward’s shape, not its level. On the scheduled MPC the fixed reward (3) with the gate reaches the same level as the agent’s rewards (+1.2 to +2.2 against +1.5 to +2.1), so the agent buys no level on that base. What it buys is load holding: the strongest fixed-reward seed loses **5.9 points of tower fatigue at 20 m/s** on the fresh winds, where all

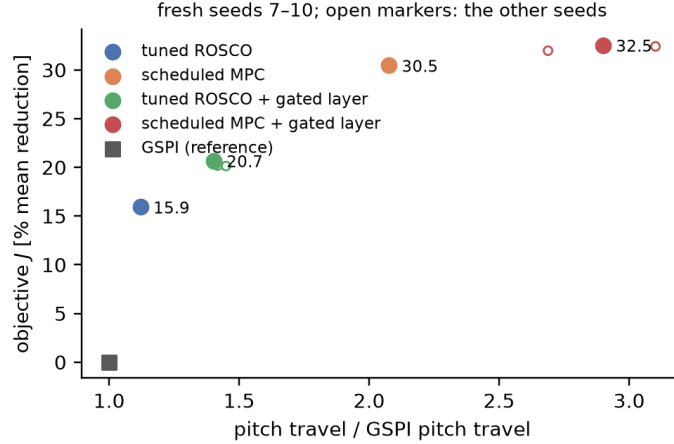


Figure 4: J against pitch travel on the fresh seeds. Filled markers: the controllers of Table 2 (best seed); open markers: the remaining seeds of each learned arm. The model-free configuration reaches 20.7 at $1.40\times$ the shipped baseline’s travel.

three agent seeds stay within 2.0 points at every wind speed (last column of Table 2). The same distinction is sharper on the tuned-PI base, where the residual has more to do: the agent’s rewards reach a solution that holds every term at every wind speed in 4 of 5 seeds, and a random search over the agent’s own vocabulary of term shapes, verified by the same forks, reaches the same level in 11 runs but holds the loads in none, losing 3–5 % of tower fatigue at 16–18 % m/s.

The proposer is not the mechanism on a strong base. On the scheduled MPC the random structural search matches the agent (mean $+1.10$ against $+0.81$ over eight runs without the gate) and holds the loads equally well. Where the base already holds the loads, the agent’s specific proposals are not distinguishable from a random draw from its vocabulary; the verification loop is what makes either work. This is consistent with Chen et al. [32], who find that language models rank controller gains correctly 31.8–53.2 % of the time and conclude that simulator-based validation remains necessary—and it is why the loop keeps nothing a 30-episode fork has not confirmed. We state it as a limit on the claim: the agent’s measurable contribution is on the weaker base.

Shape against weights. The comparison that matters for an agent that writes rewards is against a *search over the weights* of a fixed reward, which is what an adaptive-weight scheme would give [31]. A fork search over the six coefficients of (3) plateaus: it never keeps a higher tower weight, because tower fatigue responds too slowly for a 30-episode fork to see it, and it cannot reach a saturated speed term or a bounded, centred fatigue term at all. The agent’s kept expressions do exactly that, and that is the difference the load-holding column measures.

The gate is what makes the layer additive. Ungated, the same arm adds $+0.81$ (agent) and $+1.10$ (random structure) on the scheduled MPC while damaging 12–14 m/s, and the fixed reward does not learn at all—its selected checkpoint is the initial policy, i.e. the base controller, in five of five seeds. Gated, the mean gain roughly doubles ($+2.0$ held out, $+1.9$ fresh), the fixed reward finally learns, and on the tuned baseline the same gain as the ungated layer is obtained with a third less added actuation (1.40 – $1.45\times$ against 1.29 – $1.61\times$).

Learning is not the right tool for model error. A residual trained across C_P/C_T errors in $[0.85, 1.15]$ repairs only the underestimate side, adds nothing at $+15\%$, and costs 3–6 points of J at the exact model; residuals trained at one error do not transfer to another. The controller’s own offset-free and RLS estimators are 10–40 points above every residual at every error and flat over $\pm 15\%$. Learned model repair for this turbine has been reported offline [21]; the comparison here is the online one, and on this objective it favours the estimator. On a compensated base the residual has nothing of that size left to learn, and five of six runs on a mismatched base never beat their initial policy. The learned layer’s role here is not model repair; it is the part of the envelope the model-based design does not reach.

The tower weight is not a substitute for the schedule. Raising the compensated MPC’s tower weight until its high-wind tower term is non-negative costs 2.2 points of J and 20 % more pitch travel, and larger weights destabilise 12 m/s; under a per-wind load constraint the nominal controller (28.5) beats the constrained compensated one (27.3). Scheduling the weight on the wind estimate is what resolves it.

6 Discussion and limitations

The ladder answers the question the introduction posed. Of the margin a learned pitch controller would report against a shipped GSPI on this protocol, 15.4 points are tuning, another 13 are available to a correctly scaled model-based design, and what is left for learning on top of a strong base is 1.5–2.1 points—reliably, on fresh winds, but small. On a base that is only tuned, the learned layer is worth 4.2–4.8 points at $1.40\times$ pitch travel and needs no aerodynamic model, which is the configuration we would defend for deployment.

Limitations. (i) Collective pitch only; an individual-pitch version of the same layer was evaluated and is a negative result, which bounds the supervision claim rather than extending it. (ii) One turbine, one turbulence class, one 600 s episode length, seven mean wind speeds; the fresh-seed replication controls for wind realisation, not for plant or class. (iii) The agent’s contribution is established on the tuned-PI base and is not distinguishable from a random structural search on the model-based base. (iv) Everything is simulation; the pitch duty we report as the cost of each layer is an actuator-life proxy, not an actuator model. (v) The Rayleigh aggregation covers only the evaluated above-rated bins, so it is an above-rated equivalent load and its *ratio*, not its level, is the meaningful quantity.

A The per-wind constraint and the tuned fixed reward

The training objective is the per-wind variant of (1),

$$C_w = J - 20 \sum_{u \in \mathcal{U}} \sum_m \max(0, -1 - \Delta m_u [\%]), \quad (4)$$

over the seven mean wind speeds $\mathcal{U}$ and the four metrics m , each evaluated against the run’s own base controller on the same episodes: a metric that falls more than 1 % below the base at any single wind speed is charged, so a controller cannot pay for a gain at one wind speed with a loss at another. The set-mean version of the same constraint is not sufficient—a residual trained under it improves tower fatigue by 13.5 % at 8 m/s while losing 17.5 % at 15 m/s and still scores well—which is why the per-wind form is used throughout.

The coefficients of the fixed reward (3) are $w_P = 220$, $w_\omega = 25.2$, $\lambda_T = 1.0$, $\lambda_B = 1.31$, $\lambda_A = 0.1$ with $\kappa = 0.1$ rad, and the residual bounds are 0.0744 rad below rated and 0.05 rad above. They come from one fork search over the six knobs on the supervisor winds, summarised as the geometric median of the three best-checkpoint knob vectors, and are held fixed in every run that uses this reward, including every ablation.

B Controller and training details

Tuned baseline. A grid over the speed-filter corner frequency and a multiplier on the proportional gain, scored by (1) on the supervisor winds only: 3.42 rad/s (shipped 1.57) and $\times 1.13$.

MPC. Three states, horizon 2 s, re-linearised every 0.1 s, OSQP [38] at 3.6 ms per solve; cost terms normalised to $O(1)$; output-disturbance estimate with a 5 s time constant on the speed channel; tower weight 3 below 18 m/s, ramping to 6 by 22 m/s on the low-passed wind estimate. Weights and schedule selected on the supervisor winds by (1).

Residual. PPO with $\gamma = 0.998$, GAE $\lambda = 0.98$, 64×64 tanh actor and critic, running return-scale standardisation of the critic target, 300 episodes of 150 s per run, eight parallel OpenFAST workers. The supervisor decides every 30 episodes (five decisions per run); each decision trains $K = 3$ candidate rewards for a 30-episode fork from the current checkpoint, evaluates the forks on the supervisor winds, and keeps the best. The reported checkpoint is always the best by C_w on the supervisor winds; the last checkpoint is frequently degraded and is never used.

C Reward expressions written by the agent

The expressions kept by the best run on each base, in the sandbox’s variables (region: the router’s region, 1 above rated; region_w: the wind label; d_wg: relative generator-speed error; p_ratio: power over paired base power; load_t, load_b: the fatigue proxies; act: actuation). On the tuned baseline (fresh-seed J 20.68):

```
45*tanh(p_ratio-1)*(1-region)
- region_w*(36*tanh(abs(d_wg)/.004) + 10*tanh(abs(p_ratio-1)))
- (1+2*region_w)*tanh(max(load_t-1,0)) - (1+region_w)*tanh(max(load_b-1,0))
- .06*tanh(act)
```

and on the scheduled MPC (fresh-seed J 32.54):

$$40 * \tanh(8 * (p_ratio - 1)) * (1 - region) - 22 * \tanh(180 * \text{abs}(d_wg)) * region_w \\ - 10 * \tanh(8 * \text{abs}(p_ratio - 1)) * region_w \\ - (1 + region) * \tanh(\max(load_t - 1, 0)) - 1.2 * \tanh(\max(load_b - 1, 0)) - .2 * \tanh(act)$$

Three features recur and none of them is a coefficient of (3): the regulation terms are *saturated*, so a single bad gust cannot dominate an episode’s return; the fatigue terms are *one-sided and bounded*, charging only the excess of the proxy over its baseline value and saturating it; and the tower term’s weight is itself a function of the wind label. The second is what the load-holding column of Table 2 measures, and it is the reason a search over the weights of (3) cannot reach the same solution.

D Compute and reproducibility

All simulations run OpenFAST 4.2.1 with ROSCO 2.10.5 through a patched ZeroMQ interface at a 10 ms control step, on one 16-thread AMD Ryzen 7 6800H under WSL2, eight OpenFAST workers per training run. A 300-episode training run with five supervised decisions and their forks takes 2–3 h; the campaign reported in Table 2 is eight training runs and 16 evaluations of 28 episodes of 600 s, about 20 h wall-clock in total. Evaluations of deterministic controllers are exactly reproducible given the wind fields; training runs are reproducible given the seed, and we report every seed rather than a mean over few. [\[artefact and data-availability statement: repository URL, the wind-field generation script and the evaluation artefacts\]](#)

References

- [1] Ervin A. Bossanyi. The design of closed loop controllers for wind turbines. *Wind Energy*, 3(3):149–163, 2000. doi: 10.1002/we.34.
- [2] Nikhar J. Abbas, Daniel S. Zalkind, Lucy Pao, and Alan Wright. A reference open-source controller for fixed and floating offshore wind turbines. *Wind Energy Science*, 7:53–73, 2022.
- [3] Mostafa Soliman, Om P. Malik, and David T. Westwick. Multiple model predictive control for wind turbines with doubly fed induction generators. *IEEE Transactions on Sustainable Energy*, 2(3):215–225, 2011. doi: 10.1109/TSTE.2011.2153217.
- [4] Arne Koerber and Rudibert King. Combined feedback–feedforward control of wind turbines using state-constrained model predictive control. *IEEE Transactions on Control Systems Technology*, 21(4):1117–1128, 2013.
- [5] David Schlipf, Dominik Johannes Schlipf, and Martin Kühn. Nonlinear model predictive control of wind turbines using LIDAR. *Wind Energy*, 16:1107–1129, 2013. doi: 10.1002/we.1533.
- [6] Martin A. Evans, Mark Cannon, and Basil Kouvaritakis. Robust MPC tower damping for variable speed wind turbines. *IEEE Transactions on Control Systems Technology*, 23:290–296, 2015.
- [7] Sebastiaan P. Mulders et al. Preventing wind turbine tower natural frequency excitation with a quasi-LPV model predictive control scheme. *Wind Energy*, 2020. doi: 10.1002/we.2447.
- [8] Jesús Enrique Sierra-García and Matilde Santos. Exploring reward strategies for wind turbine pitch control by reinforcement learning. *Applied Sciences*, 10(21):7462, 2020. doi: 10.3390/app10217462.
- [9] Jingjie Xie, Hongyang Dong, and Xiaowei Zhao. Data-driven torque and pitch control of wind turbines via reinforcement learning. *Renewable Energy*, 215:118893, 2023.
- [10] Maud Coquelet et al. A reinforcement-learning approach for individual pitch control. *Wind Energy*, 2022. doi: 10.1002/we.2734.
- [11] Miao Wang, Hongyang Dong, and Xiaowei Zhao. A hybrid interpretable reinforcement learning framework for robust pitch control of large-scale wind turbines. *IEEE Transactions on Sustainable Energy*, 2026. In press; volume, pages and DOI to be inserted.
- [12] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.12931.
- [13] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, et al. Language to Rewards for robotic skill synthesis. In *Conference on Robot Learning (CoRL)*, 2023. arXiv:2306.08647.

- [14] Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2Reward: Reward shaping with language models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.11489.
- [15] Yecheng Jason Ma et al. DrEureka: Language model guided sim-to-real transfer. In *Robotics: Science and Systems (RSS)*, 2024. arXiv:2406.01967.
- [16] Hojoon Lee et al. RDA: Reward design agent for reinforcement learning, 2026. arXiv:2606.01672.
- [17] Andreas Klein, Thorben Wintermeyer-Kallen, János Zierath, Kevin Kluge, Dirk Abel, Heike Vallery, and Maximilian Basler. Design and practical evaluation of robust model predictive wind turbine control. *Wind Energy*, 28(5), 2025. doi: 10.1002/we.70011.
- [18] Thorben Wintermeyer-Kallen, Maximilian Basler, Thomas Konrad, János Zierath, and Dirk Abel. Challenges of applying model-based predictive wind turbine control in the field. *Forschung im Ingenieurwesen*, 87:119–128, 2023. doi: 10.1007/s10010-023-00634-1.
- [19] Thorben Wintermeyer-Kallen, Sebastian Dickler, János Zierath, Thomas Konrad, and Dirk Abel. Weight-scheduling for linear time-variant model predictive wind turbine control toward field testing. *Forschung im Ingenieurwesen*, 85:385–394, 2021. doi: 10.1007/s10010-021-00475-w.
- [20] Xiangjie Liu, Shifan Guo, Xiaobing Kong, Lele Ma, and Kwang Y. Lee. Offset-free stochastic MPC for uncertain wind energy conversion system. *IEEE Transactions on Industrial Informatics*, 20(7):9487–9496, 2024. doi: 10.1109/TII.2024.3384525.
- [21] Abhinav Anand and Carlo L. Bottasso. Adaptive economic wind turbine control. *Wind Energy Science*, 11: 1989–2008, 2026. doi: 10.5194/wes-11-1989-2026.
- [22] Manuel Lara, Sebastiaan Paul Mulders, Jan-Willem van Wingerden, Francisco Vázquez, and Juan Garrido. Simultaneous tuning of collective and individual pitch controllers for blade load reduction and power regulation in offshore wind turbines. *Ocean Engineering*, 2026. doi: 10.1016/j.oceaneng.2026.127761.
- [23] Ammar N. Abbas, Georgios C. Chasparis, and John D. Kelleher. Specialized deep residual policy reinforcement learning framework for safe and adaptive continuous control. *IET Control Theory & Applications*, 20(1), 2026. doi: 10.1049/cth2.70099.
- [24] Ammar N. Abbas, Georgios C. Chasparis, and John D. Kelleher. Specialized deep residual policy safe reinforcement learning-based controller for complex and continuous state-action spaces, 2023.
- [25] Jihong Kim, Joonhyuk Kwon, Hwa Soo Kim, TaeWon Seo, and Hyung-Tae Seo. Stability-aware residual reinforcement learning framework for robotic manipulator disturbance compensation, 2026.
- [26] Hongpeng Cao, Liqun Zhao, Yuliang Gu, Naira Hovakimyan, Lui Sha, and Marco Caccamo. Safe online learning via smooth safety-structured policy composition, 2026.
- [27] Franklin Cardenoso and Wouter Caarls. Leveraging LLMs for reward function design in reinforcement learning control tasks, 2025.
- [28] Walter P. Casas, Nelson L. S. da Fonseca, and Carlos A. Astudillo. LLM-driven automated reward design for reinforcement learning-based routing in LEO satellite networks, 2026.
- [29] Xingang Guo, Darioush Keivan, Usman Syed, Lianhui Qin, Huan Zhang, Geir Dullerud, Peter Seiler, and Bin Hu. ControlAgent: Automating control system design via novel integration of LLM agents and domain expertise, 2024.
- [30] Adil Rasheed, Oscar Ravik, and Omer San. Large language models for control, 2025.
- [31] Zhenxing Wu, Xiaoqun Fu, Sihui Li, Jinqing Peng, Hongqiang Li, and Houpei Li. A hierarchical control framework for photovoltaic-driven air conditioning systems: LLM-based reward weight adaptation and TD3 real-time control. *Building Simulation*, 2026. doi: 10.1007/s12273-026-1467-3.
- [32] Jiaxuan Chen, Yang Shu, and Hao-Nan Li. Benchmarking performance judgment in open-weight LLM controller tuning: Control knowledge does not ensure reliable gain ranking. *Processes*, 14(18):2954, 2026. doi: 10.3390/pr14182954.
- [33] International Electrotechnical Commission. IEC 61400-1: Wind energy generation systems – part 1: Design requirements. edition to be fixed at submission.
- [34] Atindriyo Kusumo Pamososuryo, Fabio Spagnolo, and Sebastiaan P. Mulders. Analysis and calibration of optimal power balance rotor-effective wind speed estimation schemes for large-scale wind turbines. *Wind Energy Science*, 10:987–1006, 2025. doi: 10.5194/wes-10-987-2025.

- [35] Jason Jonkman, Sandy Butterfield, Walter Musial, and George Scott. Definition of a 5-MW reference wind turbine for offshore system development. Technical Report NREL/TP-500-38060, National Renewable Energy Laboratory, 2009.
- [36] National Renewable Energy Laboratory. OpenFAST: an open-source wind turbine simulation tool. <https://github.com/OpenFAST/openfast>. version 4.2.1.
- [37] Gunnstein T. Frøseth. fatpack: fatigue analysis in Python. <https://github.com/Gunnstein/fatpack>.
- [38] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: an operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020. doi: 10.1007/s12532-020-00179-2.
- [39] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.
- [40] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016. arXiv:1506.02438.
- [41] Hado van Hasselt, Arthur Guez, Matteo Hessel, Volodymyr Mnih, and David Silver. Learning values across many orders of magnitude. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1602.07714.